

SIH26153

NetWatch — Technical Architecture & Operations Guide

AI-Based Network Attack Forecasting from Network Traffic Data
Counterfactual Cyber World Model · Problem Owner: NTRO

Complete reference: module internals, TShark integration, live capture pipeline,
URL destination monitoring, dashboard navigation, and end-to-end runbook.

1. System Architecture

NetWatch is an offline, AI-driven network attack forecasting platform. It ingests network traffic (PCAP/PCAPNG, CSV, JSONL), builds per-window network states, and uses a trained LSTM World Model plus an ensemble of detection engines to forecast attack stages. A counterfactual simulator evaluates defensive actions and recommends the one that minimises future risk. All computation runs locally — no cloud inference.

Layer	Components	Responsibility
Ingestion	PCAP/CSV/JSONL parsers, ZIP inspector	Normalize traffic → feature windows → network states
State Builder	netwatch/stage_classifier.py, state_manager.py	Sliding windows, host/flow aggregation, MITRE stage labeling
World Model	LSTM (PyTorch), scaler, ensemble baselines (ISO/LOF/OC-SVM)	Next-window risk + stage probability forecast
Ensemble	netwatch/ensemble.py (logreg, rf, gb, mlp)	Consensus detection across ML engines
Counterfactual	netwatch/counterfactual.py (graph sim + action scorer)	What-if simulation, action recommendation
Explainability	SHAP (TreeSHAP / DeepSHAP)	Feature attributions for risk predictions
Live Capture	netwatch/live/ — manager, pipeline, url_monitor, url_target	TShark-driven real-time capture & URL filtering
Dashboard	Flask + React (Vite), SSE for live updates	Visualization, model mgmt, traffic upload, live monitor

2. Module Reference (netwatch/)

[api/main.py](#)

Flask application factory. Registers all blueprints: upload, dashboard, models, live, forecast, graph, mitre, explainability, counterfactual, evaluation, scenarios, test_center, system, report. Serves React build from frontend/dist in production.

[api/upload.py](#)

Handles traffic ingestion (PCAP/PCAPNG/CSV/JSONL/ZIP). Validates, extracts, runs pipeline, stores session artifacts in data/data/sessions/. Returns session metadata for dashboard.

[api/dashboard.py](#)

Dashboard overview endpoints: current risk, stage, forecast summary, topology, ensemble scores, recommendations.

[api/models.py](#)

Model management: upload netwatch_trained_models.zip, list artifacts, reload pipeline, health check.

[api/live.py](#)

Legacy live monitor (interface-only). `/api/live/start`, `/status`, `/stop`, `/events` (SSE). Captures all traffic on an interface.

`api/forecast.py`

Forecast radar & projection endpoints. Returns future risk timeline, stage probabilities, confidence bands.

`api/graph.py`

Attack graph: current + predicted nodes/edges, stage transitions.

`api/mitre.py`

MITRE ATT&CK; trajectory mapping. Stage → technique mapping, kill-chain progression.

`api/explainability.py`

SHAP explanations: global feature importance, per-prediction waterfall.

`api/counterfactual.py`

What-if simulation: enumerate defensive actions, simulate graph, score risk reduction, return recommended action.

`api/evaluation.py`

Model evaluation: world model vs baselines (ISO/LOF/OC-SVM/LogReg/RF/GB/MLP) on holdout data.

`api/scenarios.py`

Ad-hoc scenario runner: inject custom traffic, run pipeline, inspect results.

`api/test_center.py`

Model test center: validation checks, data quality, drift detection.

`api/system.py`

System health: TShark availability, disk, memory, pipeline status.

`api/report.py`

Export session report (JSON/PDF).

`pipeline.py`

Core orchestrator: `run_pipeline()` — ingest → build states → forecast → ensemble → counterfactual → explainability. Returns `PipelineResult`.

`stage_classifier.py`

Heuristic + ML stage classifier. Maps feature windows → MITRE stages (RECON, INITIAL_ACCESS, EXECUTION, PERSISTENCE, PRIVILEGE_ESCALATION, DEFENSE_EVASION, CREDENTIAL_ACCESS, DISCOVERY, LATERAL_MOVEMENT, COLLECTION, COMMAND_AND_CONTROL, EXFILTRATION, IMPACT).

state_manager.py

Manages network state windows: host/flow aggregation, feature extraction, sliding window logic.

ensemble.py

Ensemble detector: trains logistic regression, random forest, gradient boosting, MLP on labeled windows. Produces consensus score.

counterfactual.py

Graph-based counterfactual simulator. Actions: BLOCK_IP, ISOLATE_HOST, RATE_LIMIT, PATCH_VULN, DEPLOY_HONEYPOT, UPDATE_RULES. Simulates edge removal/addition, scores risk delta.

world_model.py

LSTM World Model (PyTorch). Sequence → next-window risk + stage distribution. Trained in Colab, loaded at runtime.

features.py

Feature engineering: statistical, temporal, graph, protocol, entropy features per window.

data_loader.py

PCAP/CSV/JSONL loaders. Uses scapy (optional) for PCAP; pandas for CSV/JSONL.

live/manager.py

Live session lifecycle: start/stop/status/history. Tracks TShark processes, sensor metadata, world_model_status.

live/live_state.py

In-memory live state: host tracks, flow tracks, timeline events, URL telemetry, metrics history. Thread-safe with locks.

live/event_parser.py

TShark JSON parser. Extracts IP/TCP/UDP/TLS/DNS fields, computes IAT, packet sizes, flags, handshakes. Emits structured events.

live/live_pipeline.py

Real-time feature builder. Consumes parsed events → updates host/flow tracks → computes window features → calls world_model/ensemble → publishes SSE events (network_state, risk, forecast, stage, graph, mitre, explainability, counterfactual).

live/url_target.py

NEW URL resolution & tracking. UrlResolver.resolve() → A/AAAA records, ports, scheme, path. UriTargetTracker maintains IP history, resolution count, change detection.

live/url_monitor.py

NEW Destination-observed capture. Builds TShark display filter for resolved IPs + ports. Spawns TShark with -Y filter, pipes JSON to event_parser. Handles HTTPS (TLS-only visibility) and DNS change re-resolution.

live/config.py

Live capture configuration dataclasses: LiveConfig (interface, window/step size, forecast horizon), UrlMonitorConfig (url, interface, resolver settings).

3. TShark Integration — Deep Dive

TShark (Wireshark CLI) is the packet capture engine. NetWatch uses it in two modes:

- **Interface Mode (legacy)**: Captures ALL traffic on an interface. Filter: none (or optional BPF).
- **URL Monitor Mode (new)**: Captures ONLY traffic to/from resolved target IPs/ports. Filter: dynamic display filter built from UrlTargetTracker.

3.1 TShark Command Construction

In `netwatch/live/url_monitor.py`, `UrlMonitor._build_tshark_cmd()` constructs:

```
tshark -i -T json -Y "" -l
```

Where `is` built as:

```
(ip.addr == 1.2.3.4 || ip.addr == 5.6.7.8) && (tcp.port == 443 || tcp.port == 80 ||  
udp.port == 443 || udp.port == 80)
```

- `-i` = interface name (e.g., Ethernet, Wi-Fi, `eth0`)
- `-T json` = output each packet as JSON (one line per packet)
- `-Y` = display filter (applied after capture, more expressive than BPF)
- `-l` = line-buffered stdout (critical for real-time streaming)
- `-n` = no name resolution (speed)
- `-Q` = quiet (suppress packet count on stderr)

3.2 JSON Output Parsing

Each TShark JSON line contains `_source.layers` with protocols: `frame`, `eth`, `ip`, `tcp`, `udp`, `tls`, `dns`, `http`, etc. `event_parser.py` extracts:

- 5-tuple: `src_ip`, `dst_ip`, `src_port`, `dst_port`, `protocol`
- Timestamps: `frame.time_epoch` (float seconds)
- Lengths: `frame.len`, `ip.len`, TCP payload size
- TCP flags: SYN, SYN-ACK, ACK, RST, FIN, PSH, URG
- TLS: handshake type (Client Hello, Server Hello, Certificate, Finished)
- DNS: query name, response IPs (A/AAAA)

3.3 Event Emission

Parsed packets are grouped into flows (5-tuple). For each flow, the parser emits granular events to LiveState:

Event Type	Trigger	Payload
dns	DNS query/response for target hostname	hostname, resolved_ips, query_type
connection	TCP SYN (pure) to target IP:port	flow_id, direction, flags
traffic	Any packet matching target filter	bytes, flags, payload_len, tls_info
flow	New 5-tuple flow created	flow_id, protocol, endpoints
network_state	Every window step (default 5s)	features dict, timestamp
risk	World model inference	current_risk, confidence

forecast	World model forecast	future_risk[], stage_probs[]
stage	Stage classifier output	stage, probability

3.4 TShark Requirements & Troubleshooting

- **Install:** Wireshark (includes TShark). On Windows: winget install WiresharkFoundation.Wireshark. On Linux: apt install tshark / dnf install wireshark-cli.
- **Path:** NetWatch auto-detects tshark on PATH. Override with env NETWATCH_TSHARK_PATH.
- **Permissions:** Capture requires admin/root or setcap cap_net_raw,cap_net_admin=eip /usr/bin/dumpcap (Linux). Windows: run as Administrator or grant 'Capture' privilege.
- **Version:** Tested with TShark 3.x–4.x. JSON output format stable across versions.
- **No TShark?** Dashboard shows warning; live features disabled. Offline PCAP analysis still works.

4. URL Monitor — Destination-Observed Live Capture

The URL Monitor resolves a URL (e.g., `https://api.example.com/v1/data`) to its destination infrastructure and observes **ONLY** the local network transmission to/from those resolved endpoints. This is a **network monitor**, not a web scraper.

4.1 Flow: Start → Resolve → Capture → Analyze

1. **User Input:** Enter URL + select interface on `/url-monitor` page.
2. **Validate:** Client-side URL parse; backend re-validates.
3. **Resolve DNS:** `UrlResolver.resolve()` → A/AAAA records, scheme, port, path. Cached with TTL.
4. **Build Filter:** `UrlMonitor._build_display_filter()` creates TShark -Y filter for all resolved IPs + ports.
5. **Spawn TShark:** Subprocess with line-buffered JSON output. Stdout → parser thread.
6. **Parse & Track:** `event_parser` extracts packets → LiveState updates host/flow tracks, URL telemetry.
7. **Real-time Features:** `live_pipeline` computes window features → World Model + Ensemble → SSE events.
8. **DNS Re-check:** Periodic re-resolution (configurable). On IP change: filter rebuilt, TShark restarted seamlessly.
9. **Stop:** User clicks STOP or API `/api/live/url/stop` → TShark terminated, session archived.

4.2 API Endpoints

Method + Path	Request Body	Response
POST <code>/api/live/url/start</code>	<code>{url, interface, window_size?, step_size?, forecast_horizon?}</code>	<code>{analysis_id, mode, status, target, sensor}</code>
GET <code>/api/live/url/status/</code>	—	Full status: source, target, traffic, url_metrics, timeline, network_state, risk, forecast, stage, graph, mitre, explainability, counterfactual, ensemble
POST <code>/api/live/url/stop</code>	<code>{analysis_id?}</code>	<code>{status, analysis_id, mode, sensor_stats}</code>
GET <code>/api/live/events</code>	SSE	Event streams: update, url_status, dns, connection, traffic, flow, network_state, risk, forecast, stage, graph, mitre, explainability, counterfactual, stopped

4.3 HTTPS / TLS Limitations

- **Encrypted payload:** TLS 1.2/1.3 encrypts application data. NetWatch sees only metadata: handshake, record lengths, timing.
- **SNI:** Client Hello may reveal hostname (if not encrypted via ECH).
- **No MITM:** NetWatch does NOT perform TLS interception, certificate injection, or key logging.
- **What you get:** Connection counts, handshake success/failure, byte/packet rates, retransmissions, RSTs, IAT, packet sizes, flow duration — all from observable wire data.

4.4 DNS Change Handling

`UrlTargetTracker` stores `ip_history`: `List[{resolved_at, ips}]`. On each re-resolution: if new IPs differ from previous, `target_ip_changes` increments, filter rebuilt, TShark restarted. Dashboard shows **TARGET IPs UPDATED**

banner with before/after counts.

5. Live Dashboard & URL Monitor UI

5.1 Accessing the Dashboards

After starting the server (python run.py), open **http://localhost:5000**.

- **Legacy Live Monitor:** Sidebar → **Live Monitor** (/live) — captures all interface traffic.
- **URL Monitor:** Sidebar → **URL Monitor** (/url-monitor) — destination-observed capture.

5.2 URL Monitor Page Walkthrough

Header: Mode label 'URL Transmission Monitor', START/STOP button. START validates TShark availability.

Config Card: URL input (must include scheme), Interface dropdown (auto-populated from /api/live/interfaces), Mode badge (URL MONITOR). TShark status dot + version.

Target Card: Resolved destination: URL, hostname, scheme, port, resolved IP count. Capture/TShark status pills.

Monitoring Scope Card: Explicit notice: monitoring source interface, target hostname, HTTPS limitation note (if HTTPS), DNS change indicator.

Real-time Metrics (8 cards): Upload/Download rate, Packets, Bytes, Active flows, TCP connections (SYN/TLS), RST/Retransmissions, Network states (world model readiness).

Rate Chart: SVG sparkline: Packets/sec (green) + Bytes/sec (teal). Range selector: 30s / 60s / 5m. Updates via SSE + 2.5s polling.

No-Traffic State: When traffic.packets == 0: 'Monitoring active. No matching traffic observed yet.'

Network Behavior Table: Packet rate, Byte rate, IAT mean/variance, Avg packet size/variance, TCP handshake SYN/SYN-ACK, RST count, Retransmissions, TLS handshakes, Destination IPs observed, DNS resolutions.

AI Forecast Panel: Current risk %, Future max risk %, Trend, Confidence, Risk gauge bar. Predicted future state t+1..t+5 (risk %, stage, color-coded). Shows 'INSUFFICIENT DATA' until world model warms up.

Live Timeline: Reverse-chronological events from SSE: dns (blue), connection/traffic (green), flow (teal), risk (amber), forecast (red). Colored dots, timestamp, type, message.

Resolved IPs Panel: Pill badges for each resolved IP. Updates live on DNS change.

5.3 Live Events (SSE) Channel Reference

Event	Frequency	Content
update	~1-5s (batch)	Full status dict: source, target, traffic, url_metrics, timeline[]
url_status	Per packet batch	traffic, url_metrics delta
dns	On resolution	hostname, ips, query_type
connection	On TCP SYN	flow_id, direction, flags
traffic	Per packet	bytes, flags, tls, payload
flow	New flow	flow_id, protocol, endpoints
network_state	Every window step	features[], timestamp
risk	On inference	current_risk, confidence
forecast	On forecast	future[], horizon

stage	On classification	stage, probability
graph/mitre/explainability/counterfactual	On computation	Full objects
stopped	On stop	{analysis_id, reason}

6. Frontend Architecture (React + Vite)

6.1 Project Structure

```
frontend/src/
```

components/ui/ — primitives (Card, Grid, Dot, Pill, Bar, MetricCard, Button, etc.)

components/layout/ — Layout, Sidebar, TopBar

pages/ — one file per route (Overview, AnalyzePCAP, LiveMonitor, **UrlMonitor**, NetworkState, Forecast, AttackGraph, Mitre, Explainability, CounterfactualLab, ModelTestCenter, Evaluation, Scenarios, History, ReportExport, System)

services/api.ts — typed fetch wrappers for all API endpoints

types/index.ts — shared TypeScript interfaces

styles/theme.ts — palette, gauges, stage colors

App.tsx — routes + lazy loading

main.tsx — entry point

6.2 Key Patterns

- **Lazy routes:** `const Page = lazy(() => import('./pages/Page'))` — code splitting.
- **SSE hook pattern:** `EventSource` in `useEffect`, `addEventListener` per event type, cleanup on unmount.
- **Polling fallback:** 2.5s interval for full status (risk/forecast/graph) when SSE drops.
- **State accumulation:** Client-side sample buffer (max 4000 points) for charts, filtered by range selector.
- **Type safety:** Strict TS, Record over any, discriminated unions for events.
- **Theming:** Central palette object, CSS variables not used — inline styles for portability.

7. End-to-End Runbook

7.1 Full Stack Startup (Development)

Terminal 1 — Backend:

```
cd C:\sih 2026\external_hackathon\venv\Scripts\activate python run.py
```

→ Serves on **http://localhost:5000** (API + React build if exists)

Terminal 2 — Frontend Dev Server (hot reload):

```
cd C:\sih 2026\external_hackathon\frontend npm run dev
```

→ Serves on **http://localhost:5173** (proxies API to :5000 via Vite proxy)

Open **http://localhost:5173** for development (hot reload). Open **http://localhost:5000** for production-like (served by Flask).

7.2 Production Build

```
cd frontend npm run build cd .. python run.py
```

Flask serves frontend/dist/index.html and /assets/* at http://localhost:5000.

7.3 Docker (Optional)

```
docker-compose up --build → Services: web (Flask+React on :5000), optional redis/postgres if configured.
```

7.4 Complete Workflow: URL Monitor Live Capture

1. **Prerequisites:** TShark installed & on PATH (or NETWATCH_TSHARK_PATH set). Admin/root for capture.
2. **Start Backend:** python run.py → http://localhost:5000
3. **Open URL Monitor:** Sidebar → URL Monitor (/url-monitor)
4. **Configure:** Enter URL (e.g., https://httpbin.org/get), pick interface (must be UP, not loopback).
5. **Start:** Click **START URL MONITOR**. Backend resolves DNS, builds filter, spawns TShark.
6. **Observe:** Real-time metrics appear. Rate chart draws. Timeline populates. Risk/forecast update every window.
7. **Generate Traffic:** From another terminal: curl https://httpbin.org/get or browse the URL.
8. **Verify:** Packets > 0, flows > 0, upload/download rates > 0, TLS handshakes counted.
9. **Stop:** Click **STOP**. Session ends. Data retained in live state for review.
10. **Export:** Use **Report Export** page (/report) to download session JSON/PDF.

7.5 Complete Workflow: Offline PCAP Analysis

1. **Start Backend:** python run.py
2. **Upload Traffic:** Dashboard → Upload page → drop PCAP/CSV/JSONL/ZIP

- 3. Auto-Pipeline:** Ingestion triggers full pipeline: states → forecast → ensemble → counterfactual → explainability
- 4. Explore:** Navigate Dashboard, Topology, Forecast Radar, Attack Graph, Counterfactual, MITRE, Explainability, Ensemble, Evaluation
- 5. Upload Models:** Models page → drop netwatch_trained_models.zip (from Colab) for real weights
- 6. Export Report:** Report Export page → download PDF/JSON

8. End-to-End Data Flow

8.1 Offline Ingestion Pipeline

1. PCAP/CSV/JSONL/ZIP → api/upload.py
2. → data_loader.py (parse → DataFrame)
3. → features.py (extract window features)
4. → state_manager.py (build NetworkState windows)
5. → stage_classifier.py (label MITRE stages)
6. → world_model.py (LSTM forecast risk + stage probs)
7. → ensemble.py (consensus detection score)
8. → counterfactual.py (simulate actions → recommend)
9. → explainability.py (SHAP attributions)
10. → PipelineResult persisted to data/data/sessions//
11. → Dashboard pages read session artifacts

8.2 Live URL Monitor Pipeline

1. User: URL + Interface → POST /api/live/url/start
2. url_target.py: resolve() → IPs, ports, scheme, path
3. url_monitor.py: build display filter → spawn TShark -Y filter -T json -l
4. event_parser.py: parse JSON → emit dns/connection/traffic/flow events
5. live_state.py: update host_tracks, flow_tracks, url_telemetry, timeline
6. live_pipeline.py: every step_size → compute window features →
7. world_model.forward() → risk + forecast
8. ensemble.predict() → consensus
9. stage_classifier.predict() → stage
10. counterfactual.simulate() → recommended action
11. explainability.shap() → attributions
12. → publish SSE events (network_state, risk, forecast, stage, graph, mitre, explainability, counterfactual)
13. Frontend: SSE + poll → update metrics, chart, timeline, forecast panel

9. Configuration Reference

	Default	Description	
TSK_SHARK_PATH	auto-detect	Full path to tshark binary	<ParagraphStyle 'Cell'>
WINDOW_SIZE	60	Feature window seconds	<ParagraphStyle 'Cell'>
SLIDE_INTERVAL	5	Sliding step seconds	<ParagraphStyle 'Cell'>
PREDICT_HORIZON	5	Future windows to forecast	<ParagraphStyle 'Cell'>
DNS_CACHE_TTL	300	DNS cache TTL (seconds)	<ParagraphStyle 'Cell'>
RE_RESOLUTION_INTERVAL	60	Periodic re-resolution interval	<ParagraphStyle 'Cell'>
BACKEND_PORT	5000	Backend port	<ParagraphStyle 'Cell'>

E	" (relative)	Frontend API base (dev/prod)	<ParagraphStyle 'Cell'>
---	--------------	------------------------------	-------------------------

10. Testing & Validation

10.1 Backend Tests

```
python -m pytest tests/ -q
```

Key live tests in tests/test_live.py:

- test_live_health — TShark detection
- test_live_interfaces — interface enumeration
- test_live_start_stop_interface_mode — legacy capture lifecycle
- test_live_pipeline_interface_mode_emits_events — SSE event emission
- test_live_pipeline_url_mode_filters_target_traffic — **URL monitor filter correctness**
- test_live_pipeline_url_mode_dns_recheck — DNS change handling
- test_live_pipeline_url_mode_https_limitation — HTTPS metadata-only note
- test_live_url_start_invalid_url — validation
- test_live_url_start_missing_tshark — graceful degradation

10.2 Frontend Checks

```
cd frontend && npm run build -- TypeScript compile + Vite bundle (must pass)
cd frontend && npm run dev -- dev server, open /url-monitor, verify no console errors
```

10.3 Manual Smoke Test Checklist

- ✓ Backend starts, GET /api/live/health returns {available: true}
- ✓ GET /api/live/interfaces returns at least one UP non-loopback interface
- ✓ URL Monitor page loads, interface dropdown populated
- ✓ Enter valid HTTPS URL → START → phase becomes 'capturing'
- ✓ Target card shows resolved IPs (>0), scheme, port
- ✓ Generate traffic (curl/browser) → packets/bytes > 0 in metrics cards
- ✓ Rate chart draws lines (packets/sec, bytes/sec)
- ✓ Timeline shows dns → connection → traffic → flow events
- ✓ Risk/forecast panel updates (may show 'INSUFFICIENT DATA' until model warms)
- ✓ STOP button ends capture, phase returns to 'idle'
- ✓ Re-start works (new analysis_id)

NetWatch · SIH26153 · Counterfactual Cyber World Model — Problem Owner: NTRO.

Generated from make_tech_guide_pdf.py. For questions, see README.md or repo issues.